

AWS DevOps Interview Top 50 Intermediate Questions with Hints

Digitalearn Solution

Intermediate Level • CI/CD • Containers • IaC • Monitoring • Networking

Q1. What is the difference between ALB, NLB, and CLB?

Hint: ALB = Layer 7 (HTTP/HTTPS, path & host routing); NLB = Layer 4 (TCP/UDP, ultra-low latency); CLB = legacy. ALB is preferred for microservices and containerized apps.

Q2. How does AWS Auto Scaling work and what are its key components?

Hint: Components: Auto Scaling Group + Launch Template + Scaling Policies. Types: Target Tracking, Step Scaling, Scheduled. Integrates with ALB for traffic distribution.

Q3. Explain Blue/Green deployment strategy on AWS.

Hint: Blue = current prod; Green = new version. Switch traffic via Route 53 weighted routing or ELB target groups. Zero-downtime; instant rollback by redirecting traffic back.

Q4. What is AWS ECS and how does it differ from EKS?

Hint: ECS = AWS-native container orchestration (simpler, tight AWS integration). EKS = managed Kubernetes (more portable, more complex). Both support EC2 and Fargate launch types.

Q5. What is AWS Fargate and when would you use it?

Hint: Fargate = serverless compute for containers. No EC2 node management. Use when you want to run containers without managing infrastructure. Works with both ECS and EKS.

Q6. How does AWS CodeDeploy work and what deployment strategies does it support?

Hint: CodeDeploy automates deployments to EC2, Lambda, and ECS. Strategies: In-place and Blue/Green. Uses AppSpec.yml to define lifecycle event hooks.

Q7. What is AWS CodeBuild and how is buildspec.yml structured?

Hint: CodeBuild = managed build service. buildspec.yml phases: install, pre_build, build, post_build. Supports env variables, caching, Docker builds, artifact uploads to S3.

Q8. Explain AWS CloudWatch and its key DevOps capabilities.

Hint: Metrics, Logs, Alarms, Dashboards, Events/EventBridge, Container Insights, Log Insights. Set alarms to trigger Auto Scaling or SNS notifications. Core observability service.

Q9. What is AWS Elastic Beanstalk and how does it simplify application deployments?

Hint: PaaS — upload code, AWS manages EC2, load balancer, Auto Scaling, and monitoring. Use .ebextensions for customization. Supports rolling, immutable, and blue/green deployments.

Q10. How does Amazon RDS Multi-AZ differ from Read Replicas?

Hint: Multi-AZ = synchronous standby for HA/failover (not for performance). Read Replicas = asynchronous copies for read scaling. Multi-AZ failover is automatic; replica promotion is manual.

Q11. What is Terraform and how does it compare to AWS CloudFormation?

Hint: Terraform = multi-cloud IaC (HCL), open-source, excellent state management. CloudFormation = AWS-only, YAML/JSON, native AWS integration. Terraform is more portable across providers.

Q12. Explain AWS SQS vs SNS vs EventBridge — when to use each?

Hint: SQS = queuing for decoupling (pull-based). SNS = pub/sub fan-out (push). EventBridge = event bus for complex routing across AWS services and SaaS. Often used together.

Q13. What is AWS Secrets Manager and how does it differ from Parameter Store?

Hint: Secrets Manager = auto-rotation, per-secret cost, designed for credentials. Parameter Store = cheaper, hierarchical config, free tier available. Both integrate with IAM for fine-grained access.

Q14. How do you implement centralized logging best practices in AWS?

Hint: CloudWatch Logs for app/service logs, VPC Flow Logs, CloudTrail for API audit, ALB access logs to S3. Centralize with CloudWatch Log Insights, OpenSearch, or third-party SIEM.

Q15. What is AWS CloudTrail and why is it critical for DevOps teams?

Hint: CloudTrail records all AWS API calls — who, what, when, from where. Critical for auditing, security investigations, compliance, and change tracking. Enable in all regions.

Q16. How do you implement rolling deployments in AWS ECS?

Hint: ECS rolling update uses 'minimumHealthyPercent' and 'maximumPercent' settings. ECS drains old tasks while starting new ones. Set health check grace period to avoid premature termination.

Q17. What is AWS ECR and how does it integrate with CI/CD pipelines?

Hint: ECR = managed Docker container registry. Push images via CodeBuild, pull in ECS/EKS deployments. Supports image scanning, lifecycle policies, and cross-account access via resource policies.

Q18. Explain the purpose of Target Groups in AWS load balancing.

Hint: Target Groups route requests to registered targets (EC2, Lambda, IP). Each has health check settings. ALB can route to different target groups based on path/host rules.

Q19. What is AWS Systems Manager (SSM) and how is it used in DevOps?

Hint: SSM = operational management service. Key features: Session Manager (no SSH), Run Command, Patch Manager, Parameter Store, Automation runbooks. Eliminates bastion hosts.

Q20. How do you implement environment-specific configurations in AWS?

Hint: Use SSM Parameter Store or Secrets Manager with environment-prefixed paths (/dev/db-url, /prod/db-url). Reference in ECS task definitions, Lambda env vars, or CodeBuild env variables.

Q21. What is AWS CodePipeline and how do you structure a multi-stage pipeline?

Hint: CodePipeline orchestrates Source → Build → Test → Deploy stages. Each stage has actions. Add manual approval gates before production. Integrates with CodeCommit, CodeBuild, CodeDeploy, ECR.

Q22. How does AWS IAM Roles for Service Accounts (IRSA) work in EKS?

Hint: IRSA maps Kubernetes service accounts to IAM roles via OIDC. Pods assume IAM roles without needing access keys on nodes. Best practice for pod-level AWS permission management.

Q23. What are AWS Launch Templates and how do they differ from Launch Configurations?

Hint: Launch Templates = newer, support versioning, Spot + On-Demand mixing, more instance types. Launch Configurations = legacy, immutable. Always use Launch Templates for new ASGs.

Q24. Explain the difference between active-active and active-passive architectures in AWS.

Hint: Active-Active = traffic served by multiple regions simultaneously (lower latency, higher cost). Active-Passive = standby region activates on failover. Route 53 routing policies handle both.

Q25. What is AWS Config and how does it support compliance in DevOps?

Hint: AWS Config = resource inventory, change history, and compliance rules. Managed and custom rules for drift detection. Integrate with Systems Manager Automation for auto-remediation.

Q26. How do you manage Docker image versioning in a CI/CD pipeline?

Hint: Tag images with Git commit SHA, branch name, or build number. Never rely solely on 'latest'. Push to ECR with semantic versions. Reference exact image tags in ECS task definitions.

Q27. What is the purpose of health checks in AWS and how do they differ across services?

Hint: ALB health checks route traffic only to healthy targets. Auto Scaling uses EC2 status checks or ELB checks for replacement. Route 53 health checks drive DNS failover.

Q28. Explain how VPC peering works and its limitations.

Hint: VPC peering = private connectivity between VPCs (same or different accounts/regions). Limitations: not transitive, no overlapping CIDRs, no edge-to-edge routing via gateways.

Q29. What is AWS Transit Gateway and when would you use it over VPC peering?

Hint: Transit Gateway = hub-and-spoke network connectivity for many VPCs and on-prem. Use when managing 5+ VPCs — peering becomes unmanageable. Supports transitive routing unlike peering.

Q30. How do you implement automated patching in AWS?

Hint: Use AWS Systems Manager Patch Manager with Patch Baselines and Maintenance Windows. Define patch groups via EC2 tags. Scan for compliance and install patches on a schedule.

Q31. What is AWS CloudFront and how does it fit in a DevOps pipeline?

Hint: CloudFront = CDN that caches content at edge locations globally. Reduces origin load, improves latency. Integrate with S3, ALB, or API Gateway as origins. Invalidate cache on deployments.

Q32. How do you implement infrastructure as code testing?

Hint: Use Terraform: terraform validate, terraform plan, tfint, checkov for security. For CloudFormation: cfn-lint, cfn-nag. Add IaC tests to CodePipeline before applying to production.

Q33. What is AWS Step Functions and how is it used in DevOps workflows?

Hint: Step Functions = serverless state machine orchestration. Use for multi-step workflows, retry logic, parallel processing. Integrates natively with Lambda, ECS, SQS, SNS, and DynamoDB.

Q34. How do you handle database migrations in a CI/CD pipeline on AWS?

Hint: Run migrations as a pre-deploy step using Lambda, CodeBuild, or ECS task. Use tools like Flyway or Liquibase. Ensure migrations are idempotent and backward compatible for rollback safety.

Q35. What is Amazon ElastiCache and when would you use it in a DevOps architecture?

Hint: ElastiCache = managed Redis or Memcached. Use for session caching, database query caching, real-time leaderboards. Reduces DB load. Redis supports persistence, pub/sub, and clustering.

Q36. How do you implement feature flags in AWS deployments?

Hint: Use AWS AppConfig (part of Systems Manager) for dynamic feature flag configuration. Supports deployment strategies and rollback. Alternatively use LaunchDarkly or custom SSM Parameter Store flags.

Q37. What is AWS WAF and how do you integrate it into a DevOps pipeline?

Hint: WAF = Web Application Firewall. Attach to ALB, CloudFront, or API Gateway. Define rules/rule groups for SQL injection, XSS, rate limiting. Manage WAF rules as code with Terraform.

Q38. Explain the concept of immutable infrastructure and how to achieve it on AWS.

Hint: Never modify running infrastructure — replace it. Achieve via AMI baking with Packer, ECS with new task definitions, or Lambda deployments. Ensures consistent, drift-free environments.

Q39. How do you implement cost optimization in a DevOps CI/CD workflow on AWS?

Hint: Use Spot Instances for CodeBuild/Jenkins agents, S3 lifecycle policies for artifacts, schedule non-prod environment shutdowns with Lambda, right-size EC2, and use Reserved Instances for stable workloads.

Q40. What is AWS Service Catalog and how does it help DevOps teams?

Hint: Service Catalog = curated portfolio of pre-approved CloudFormation/Terraform products. Enables self-service provisioning with guardrails. Helps enforce standards across teams at scale.

Q41. How does Amazon API Gateway integrate with Lambda in a serverless DevOps pipeline?

Hint: API Gateway = fully managed API layer. Routes HTTP requests to Lambda functions. Deploy via CDK/SAM/Terraform. Use stages (dev/prod), canary deployments, and throttling for traffic control.

Q42. What is AWS Backup and how do you automate backup policies?

Hint: AWS Backup = centralized backup service for EBS, RDS, DynamoDB, EFS, S3. Create Backup Plans with schedule, retention, and vault settings. Tag resources for automatic backup assignment.

Q43. How do you implement monitoring and alerting for a containerized workload on ECS?

Hint: Enable CloudWatch Container Insights for cluster/service/task metrics. Set alarms on CPU, memory, task count, ALB 5xx errors. Use EventBridge to trigger Lambda on ECS task failures.

Q44. What is the difference between Deployment, DaemonSet, and StatefulSet in Kubernetes (EKS)?

Hint: Deployment = stateless apps with rolling updates. DaemonSet = one pod per node (e.g., log collectors). StatefulSet = stateful apps with stable network IDs and persistent storage (e.g., databases).

Q45. How do you manage Terraform state in a team environment on AWS?

Hint: Use S3 as remote backend for state storage with DynamoDB for state locking. Enable S3 versioning for state history. Never commit .tfstate to Git. Use Terraform Cloud or Atlantis for team workflows.

Q46. What is AWS Kinesis and when would a DevOps engineer use it?

Hint: Kinesis = real-time data streaming. Kinesis Data Streams for custom processing; Kinesis Firehose for delivery to S3/Redshift/OpenSearch. Use for log streaming, real-time metrics, or event pipelines.

Q47. How do you implement role-based access control (RBAC) in EKS?

Hint: EKS uses Kubernetes RBAC (Roles/ClusterRoles + Bindings) combined with IAM. Map IAM users/roles to Kubernetes groups via aws-auth ConfigMap. Use IRSA for pod-level AWS permissions.

Q48. What is AWS Proton and how does it help platform engineering teams?

Hint: AWS Proton = managed service for platform teams to define standardized templates (environment + service). Dev teams self-serve deployments. Integrates with CodePipeline and Terraform.

Q49. How do you implement zero-trust networking in AWS?

Hint: Use IAM with least privilege, VPC security groups as micro-segmentation, PrivateLink for service access, App Mesh with mTLS for service-to-service encryption, and AWS Verified Access for app access.

Q50. What is the AWS Well-Architected Framework and how does it guide DevOps practices?

Hint: Six pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability. Use the Well-Architected Tool to assess workloads and identify improvements.
